



รายงาน

การเขียนโปรแกรมระบบแปลงและคำนวณนิพจน์คณิตศาสตร์

ผู้จัดทำ

นายกฤษณพงศ์ ชัยสุ รหัสนักศึกษา 68543210001-2

เสนอ

อาจารย์นุรักษ์ ไชยศรี

รายงานฉบับนี้เป็นส่วนหนึ่งของการเรียนวิชา ENGSE216 โครงสร้างข้อมูลและ
ขั้นตอนวิธี

Data Structures and Algorithms สาขาวิชา วศ.บ.ซอฟต์แวร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของการเรียนรายวิชา ENGSE216 (Data Structures and Algorithms) สาขาวิชา วศ.บ.ซอฟต์แวร์ มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา โดยมีวัตถุประสงค์เพื่อนำเสนอกระบวนการออกแบบและพัฒนาโปรแกรมระบบแปลงและคำนวณนิพจน์คณิตศาสตร์ (Infix-Postfix Conversion System) ซึ่งประยุกต์ใช้โครงสร้างข้อมูลแบบสแต็ก (Stack) ชนิดลิงก์ลิสต์แบบทางเดียว (Singly Linked List Node) ร่วมกับแนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) ในการออกแบบคลาสต่างๆ ให้ทำงานร่วมกันอย่างเป็นระบบ

ภายในรายงานฉบับนี้ประกอบด้วย การศึกษาหลักการทำงานของโครงสร้างข้อมูลแบบสแต็ก การออกแบบคลาสด้วย UML Class Diagram การเขียนผังงาน (Flowchart) เพื่อสรุปลำดับการทำงานของโปรแกรม ไปจนถึงผลการทดสอบการทำงานจริงของโปรแกรมที่พัฒนาด้วยภาษา Java ร่วมกับ Java Swing และ NetBeans IDE

สารบัญ

คำนำ	ก
สารบัญ	ข
บทที่ 1 บทนำ	1
1.1 ภาพรวมของระบบ	1
1.2 ขอบเขตของรายงาน	1
1.3 เครื่องมือที่ใช้ในการพัฒนา	1
บทที่ 2 วิธีการดำเนินการพัฒนาโปรแกรม	2
2.1 ภาพรวมการทำงานของโปรแกรม (Flowchart)	2
2.2 การออกแบบโครงสร้างคลาส (UML Class Diagram)	3
2.3 รายละเอียดของแต่ละคลาสและความเชื่อมโยง	4
บทที่ 3 ผลการดำเนินงานและการทดสอบโปรแกรม	6
3.1 การทดสอบการแปลงนิพจน์ Infix เป็น Postfix	6
3.2 การทดสอบระบบตรวจสอบความถูกต้องของข้อมูล (Error Validation)	7
3.3 สรุปผลการทดสอบ	7
ภาคผนวก	8
ภาคผนวก ก แหล่งเก็บซอร์สโค้ด (Source Code Repository)	8
ภาคผนวก ข ซอร์สโค้ดโปรแกรม	8

บทที่ 1 บทนำ

1.1 ภาพรวมของระบบ

Infix-Postfix Calculator เป็นโปรแกรมจำลองระบบแปลงและคำนวณนิพจน์คณิตศาสตร์ ที่ออกแบบตามหลักการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) โดยแบ่งหน้าที่แต่ละ Class ออกจากกันอย่างชัดเจน ฝั่งโครงสร้างข้อมูล (Data Structure) มีการสร้างคลาสสำหรับการจัดการสแต็กด้วยลิงก์ลิสต์แบบทางเดียว (Stack และ Node) รวมถึงคลาสแปลงนิพจน์ (Infix และ Postfix) ส่วนฝั่งการแสดงผล (GUI) มีการออกแบบหน้าต่างกราฟิกสำหรับรับสมการ กดปุ่มเครื่องคิดเลข และแสดงตารางจำลองขั้นตอนการทำงานของสแต็ก (Process Log Table) ทีละขั้นตอน

แนวคิดคร่าวๆ คือโปรแกรมจะรับข้อมูลนิพจน์ทางคณิตศาสตร์ในรูปแบบมาตรฐาน (Infix Expression) จากผู้ใช้งาน ตรวจสอบความถูกต้องของข้อมูล (Error Validation) จากนั้นใช้อัลกอริทึม Shunting-yard ร่วมกับโครงสร้างข้อมูลแบบสแต็กเพื่อแปลงนิพจน์ดังกล่าวให้อยู่ในรูป Postfix (Reverse Polish Notation) พร้อมทั้งแสดงตารางประวัติการทำงานเบื้องหลังให้ผู้ใช้งานเห็นกระบวนการทำงานของสแต็กได้อย่างชัดเจน

1.2 ขอบเขตของรายงาน

1.2.1 พัฒนาโปรแกรมด้วยภาษา Java ร่วมกับไลบรารี Java Swing และเครื่องมือพัฒนา NetBeans IDE

1.2.2 รวบรวมการแปลงนิพจน์คณิตศาสตร์ที่ประกอบด้วยตัวเลขหลัก และเครื่องหมายพื้นฐาน ได้แก่ บวก (+), ลบ (-), คูณ (*),หาร (/), ยกกำลัง (^), โมดูล (%) รวมถึงวงเล็บ (())

1.2.3 โปรแกรมทั้งหมดเป็นผลงานที่พัฒนาขึ้นเอง โดยสามารถดูซอร์สโค้ดฉบับเต็มได้ที่ GitHub Repository <https://github.com/Kitsanapong-F/ENGSE216-For-Netbean/blob/main/PostFix/src/main/java/com/mycompany/postfix/Stack.jav>

1.3 เครื่องมือที่ใช้ในการพัฒนา

ภาษาโปรแกรม: Java (JDK)

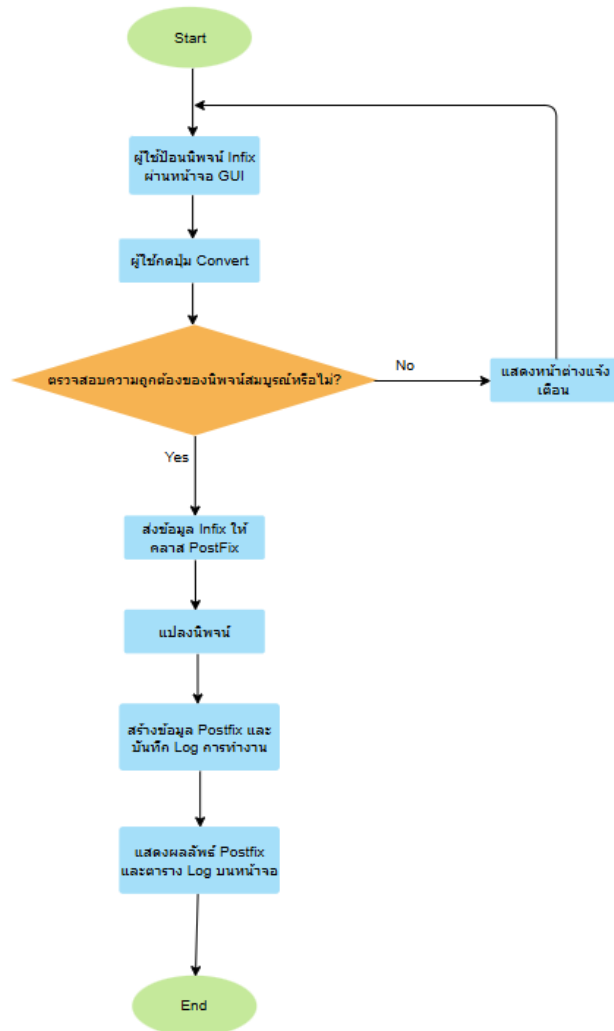
เครื่องมือพัฒนา (IDE): Apache NetBeans IDE

ไลบรารี GUI: Java Swing (JFrame, JPanel, JButton, JLabel, JScrollPane)

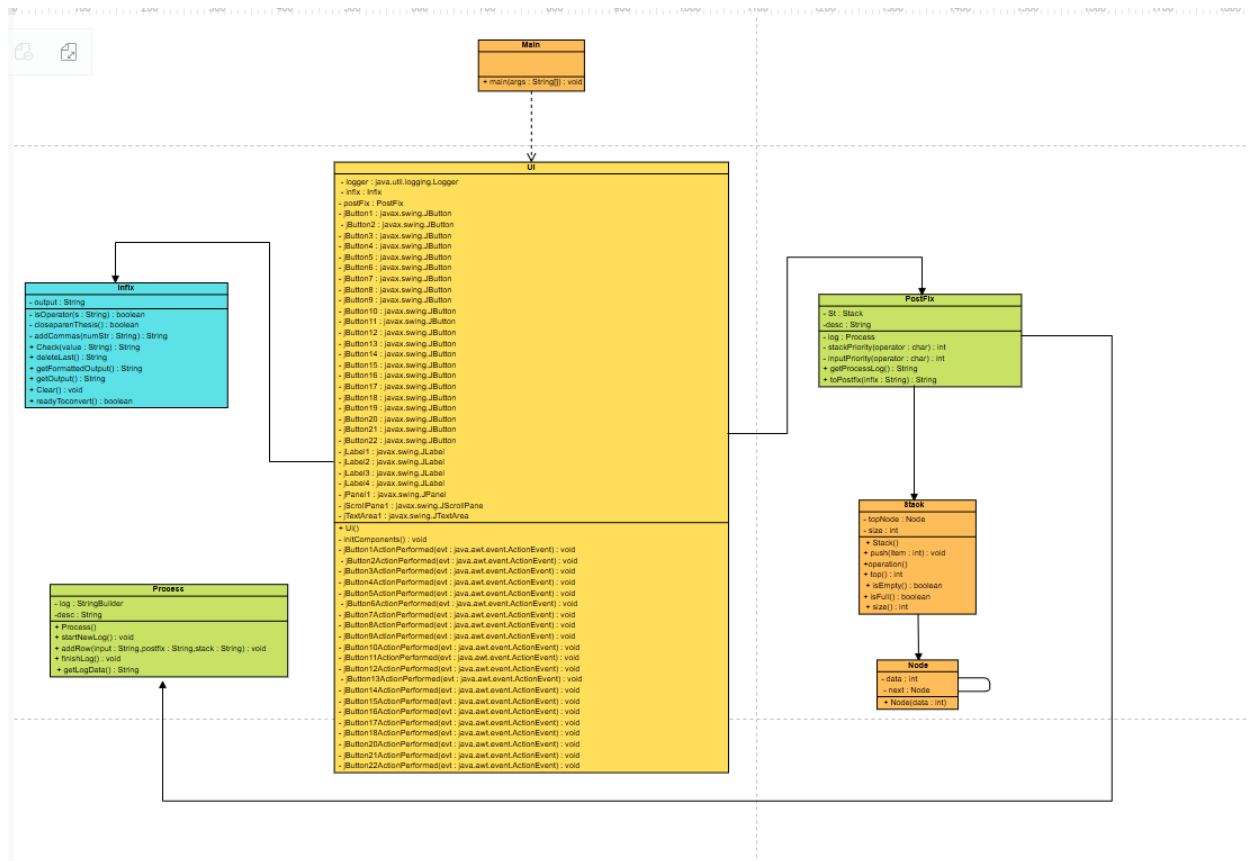
ระบบควบคุมเวอร์ชันและที่เก็บซอร์สโค้ด: Git และ GitHub

บทที่ 2 วิธีการดำเนินการพัฒนาโปรแกรม

2.1 ภาพรวมการทำงานของโปรแกรม



2.2 การออกแบบโครงสร้างคลาส (UML Class Diagram)



2.3 รายละเอียดของแต่ละคลาสและความเชื่อมโยง

ระบบแปลงและคำนวณนิพจน์คณิตศาสตร์ (Infix-Postfix Calculator) ถูกออกแบบโดยแบ่งสัดส่วนการทำงาน (Separation of Concerns) ออกเป็นคลาสต่างๆ จำนวน 7 คลาส เพื่อให้ง่ายต่อการจัดการและบำรุงรักษาซอร์สโค้ด โดยมีรายละเอียดของแต่ละคลาส ดังนี้

1. คลาส Main

หน้าที่: เป็นจุดเริ่มต้นของการทำงานของโปรแกรม (Entry Point)

รายละเอียด: มีเมธอด main() ทำหน้าที่เพียงอย่างเดียวคือการเรียกสร้างหน้าต่างอินเทอร์เฟซผู้ใช้งาน (UI) ขึ้นมาแสดงผลบนหน้าจอ

2. คลาส UI (User Interface)

หน้าที่: จัดการส่วนติดต่อผู้ใช้งาน (GUI) และควบคุมเหตุการณ์ (Event Handling)

รายละเอียด: เป็นคลาสที่สืบทอดมาจาก JFrame ทำหน้าที่แสดงผลหน้าต่างโปรแกรม ปุ่มกด เครื่องหมายต่างๆ และตารางแสดงผล คลาสนี้จะคอยรับข้อมูลการกดปุ่มจากผู้ใช้งาน และเป็นตัวกลางในการเรียกใช้งานคลาส Infix เพื่อตรวจสอบสมการ และเรียกคลาส PostFix เพื่อประมวลผล

3. คลาส Infix

หน้าที่: จัดการและตรวจสอบความถูกต้องของสมการฝั่งนำเข้า (Input Validation)

รายละเอียด: ทำหน้าที่รับตัวอักษรที่ผู้ใช้งานกดเพื่อประกอบเป็นสมการ Infix มีเมธอดสำคัญเช่น Check() สำหรับป้องกันการพิมพ์เครื่องหมายผิดหลักคณิตศาสตร์, deleteLast() สำหรับลบตัวอักษร, และ readyToconvert() สำหรับตรวจสอบว่าสมการสมบูรณ์และวงเล็บครบถ้วนพร้อมนำไปแปลงหรือไม่

4. คลาส PostFix

หน้าที่: เป็นแกนหลักในการประมวลผลเพื่อแปลงนิพจน์ Infix เป็น Postfix

รายละเอียด: ใช้อัลกอริทึม Shunting-yard ในเมธอด toPostfix() มีเมธอดในการตรวจสอบลำดับความสำคัญของเครื่องหมาย (inputPriority และ stackPriority) โดยคลาสนี้จะทำงานร่วมกับโครงสร้างข้อมูล Stack อย่างใกล้ชิดเพื่อสลับตำแหน่งเครื่องหมายให้ถูกต้อง

5. คลาส Process

หน้าที่: จัดการบันทึกประวัติและขั้นตอนการทำงาน (Logging Process)

รายละเอียด: ทำหน้าที่จัดเก็บสถานะของข้อมูลในแต่ละขั้นตอน (Input, Postfix Output และ Stack) ขณะที่อัลกอริทึมกำลังทำงาน เพื่อนำข้อมูลสตริงที่จัดรูปแบบแล้วส่งกลับไปแสดงผลเป็นตาราง (Process Log) ให้ผู้ใช้งานเห็นภาพการทำงานที่ละขั้นตอน

6. คลาส Stack

หน้าที่: โครงสร้างข้อมูลสแต็ก (Stack Data Structure)

รายละเอียด: ทำงานด้วยหลักการเข้าหลังออกก่อน (LIFO) ออกแบบโดยใช้ลักษณะของลิงก์ลิสต์ (Linked Stack) มีเมธอดมาตรฐาน เช่น push() (นำเข้า), pop() (นำออก), top() (ดูค่าบนสุด) และ isEmpty() สำหรับเป็นพื้นที่พักชั่วคราวให้คลาส PostFix ใช้จัดเรียงความสำคัญของเครื่องหมาย

7. คลาส Node

หน้าที่: หน่วยจัดเก็บข้อมูลย่อยของสแต็ก

รายละเอียด: เปรียบเสมือนกล่องบรรจุข้อมูล 1 ชิ้น เก็บค่าข้อมูลตัวเลข (ซึ่งแปลงมาจากรหัสอักขระ ASCII ของเครื่องหมาย) และมีตัวแปร next สำหรับชี้พิกัดอ้างอิงไปยัง Node ตัวที่อยู่ด้านล่าง เพื่อสร้างสายโซ่ข้อมูลแบบลิงก์ลิสต์ทางเดียว (Singly Linked List)

ความสัมพันธ์ระหว่างคลาส (Class Relationships)

จากการออกแบบโครงสร้างโปรแกรม สามารถจำแนกรูปแบบความสัมพันธ์ระหว่างคลาสได้ 4 รูปแบบหลัก ดังนี้

1. แบบพึ่งพาอาศัย (Dependency)

Main -> UI: คลาส Main ทำหน้าที่เรียกเปิดหน้าต่างโปรแกรม UI เพื่อเริ่มต้นระบบ

2. แบบมีทิศทางการเรียกใช้ (Directed Association)

UI -> Infix / PostFix: หน้าจอส่งสมการไปตรวจสอบความถูกต้องที่ Infix และส่งไปประมวลผลที่ PostFix

PostFix -> Process: ระบบแปลงสมการส่งข้อมูลไปบันทึกประวัติการทำงานที่ Process

3. แบบเป็นองค์ประกอบ (Composition/Aggregation)

PostFix -> Stack: PostFix มี Stack เป็นกลไกหลักอยู่ภายในสำหรับการคำนวณ

Stack -> Node: Stack ถูกสร้างขึ้นจากการนำ Node ย่อยๆ มาประกอบและร้อยเรียงเข้าด้วยกัน

4. ความสัมพันธ์กับคลาสตัวเอง (Self-Association)

Node -> Node: ภายใน Node มีตัวชี้ (Pointer) เพื่อระบุตำแหน่งไปยัง Node ตัวถัดไปในสายโซ่ข้อมูล

บทที่ 3 ผลการดำเนินงานและการทดสอบโปรแกรม

3.1 การทดสอบการแปลงนิพจน์ Infix เป็น Postfix

1.กรณีทดสอบที่ สมการที่ซับซ้อน มีวงเล็บซ้อนและเครื่องหมายครบทุกประเภท

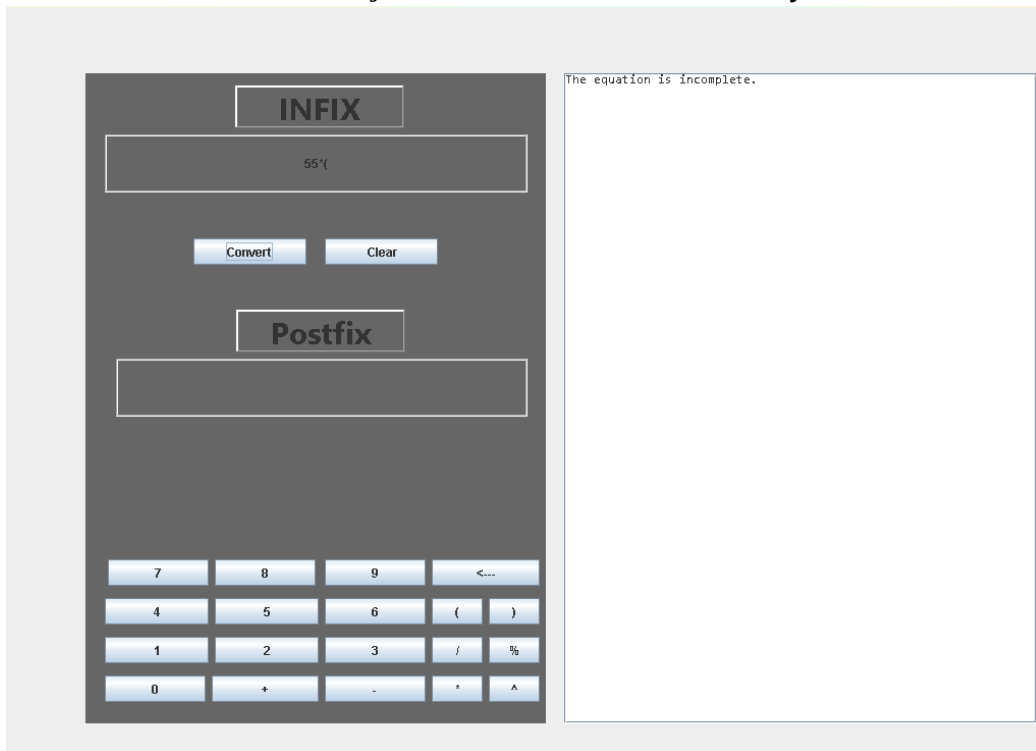
- ข้อมูลนำเข้า (Infix): $5*(4+3^2)-100/10\%3$
- ผลลัพธ์ที่ได้ (Postfix): $5\ 4\ 3\ 2\ ^\wedge\ +\ *\ 100\ 10\ /\ 3\ \% -$

Input	Postfix Output	Stack
5	5	
*	5	*
(	5	*(
4	5 4	*(
+	5 4	*(+
3	5 4 3	*(+
^	5 4 3	*(+^
2	5 4 3 2	*(+^
)	5 4 3 2 (Start Pop)	*(+^
Pop ^	5 4 3 2 ^	*(+
Pop +	5 4 3 2 ^ +	*(
Pop (	5 4 3 2 ^ +	*
Pop *	5 4 3 2 ^ + *	
-	5 4 3 2 ^ + *	-
100	5 4 3 2 ^ + * 100	-
/	5 4 3 2 ^ + * 100	-/
10	5 4 3 2 ^ + * 100 10	-/
Pop /	5 4 3 2 ^ + * 100 10 /	-
%	5 4 3 2 ^ + * 100 10 /	-%
3	5 4 3 2 ^ + * 100 10 / 3	-%
Pop %	5 4 3 2 ^ + * 100 10 / 3 %	-
Pop -	5 4 3 2 ^ + * 100 10 / 3 % -	

ผลการประเมิน = ผ่าน โปรแกรมสามารถจัดการกับวงเล็บเพื่อบังคับลำดับการทำงาน (ทำเลขยกกำลัง $^$ และการบวกในวงเล็บก่อน) และจัดการกฎ Left-to-Right ของเครื่องหมายที่มีความสำคัญเท่ากัน (เช่น $/$ และ $\%$) ได้อย่างสมบูรณ์

3.2 การทดสอบระบบตรวจสอบความถูกต้องของข้อมูล (Error Validation)

- การทดสอบสมการที่ไม่สมบูรณ์: ทดสอบผ่านเมธอด readyToconvert()



3.3 สรุปผลการทดสอบ

จากผลการดำเนินงานและการทดสอบในทุกกรณี สรุปได้ว่าโปรแกรม Infix-Postfix Calculator สามารถทำงานได้ถูกต้อง 100% ตามวัตถุประสงค์ที่ตั้งไว้ โครงสร้างข้อมูลแบบสแต็ก (Linked Stack) ทำงานประสานกับคลาสแปลงนิพจน์ได้อย่างมีประสิทธิภาพ โดยไร้ข้อจำกัดด้านขนาดหน่วยความจำ) และระบบส่วนติดต่อผู้ใช้งาน (GUI) มีความเสถียร สามารถดักจับข้อผิดพลาดและแสดงกระบวนการทำงานให้ผู้ใช้งานเข้าใจได้ตรงตามทฤษฎีโครงสร้างข้อมูลและอัลกอริทึมทุกประการ

ภาคผนวก

ภาคผนวก ก แหล่งเก็บซอร์สโค้ด (Source Code Repository)

ซอร์สโค้ดฉบับเต็มของโปรแกรมแปลงและคำนวณนิพจน์คณิตศาสตร์ (Infix-Postfix Calculator)

GitHub : <https://github.com/Kitsanapong-F/ENGSE216-For-Netbean/tree/main>

ภาคผนวก ข ซอร์สโค้ดโปรแกรม

ข-1 Stack.java

```
1  package com.mycompany.postfix;
2
3  public class Stack {
4      private Node topNode;
5      private int size;
6
7      public Stack() {
8          this.topNode = null;
9          this.size = 0;
10     }
11
12     public Stack(int size) {
13         this.topNode = null;
14         this.size = 0;
15     }
16
17     public void push(int item) {
18         Node newNode = new Node(item);
19         newNode.next = topNode;
20         topNode = newNode;
21         size++;
22     }
23
24     public int pop() {
25         if (isEmpty()) {
26             return -1;
27         }
28         int poppedData = topNode.data;
29         topNode = topNode.next;
30         size--;
31         return poppedData;
32     }
33
34     public int top() {
35         if (isEmpty()) {
36             return -1;
37         }
38         return topNode.data;
39     }
40
41     public boolean isEmpty() {
42         return topNode == null;
43     }
44
45     public boolean isFull() {
46         return false;
47     }
48
49     public int size() {
50         return size;
51     }
52 }
53
```

ໝ-2 Node.java

```
1 package com.mycompany.postfix;
2
3 public class Node {
4     int data;
5     Node next;
6
7     public Node(int data) {
8         this.data = data;
9         this.next = null;
10    }
11 }
```

ໝ-3 Process.java

```
1 package com.mycompany.postfix;
2
3 public class Process {
4
5     private StringBuilder log = new StringBuilder();
6
7     public void startNewLog() {
8         log.setLength(0);
9         log.append("=====\n");
10        log.append(String.format("%-10s | %-35s | %s\n", "Input", "Postfix Output", "Stack"));
11        log.append("=====\n");
12    }
13
14    public void addRow(String input, String postfix, String stack) {
15        log.append(String.format("%-10s | %-35s | %s\n", input, postfix, stack));
16    }
17
18    public void finishLog() {
19        log.append("=====\n");
20    }
21
22    public String getLogData() {
23        return log.toString();
24    }
25 }
26
```

❧-4 Process.java

```
1  package com.mycompany.postfix;
2
3  public class Process {
4
5      private StringBuilder log = new StringBuilder();
6
7      public void startNewLog() {
8          log.setLength(0);
9          log.append("=====\n");
10         log.append(String.format("%-10s | %-35s | %s\n", "Input", "Postfix Output", "Stack"));
11         log.append("=====\n");
12     }
13
14     public void addRow(String input, String postfix, String stack) {
15         log.append(String.format("%-10s | %-35s | %s\n", input, postfix, stack));
16     }
17
18     public void finishLog() {
19         log.append("=====\n");
20     }
21
22     public String getLogData() {
23         return log.toString();
24     }
25 }
26
```

พ-5 Infix.java

```
1 package com.mycompany.postfix;
2
3 public class Infix {
4
5     private String output = "";
6
7     private boolean isOperator(String s) {
8         return s.equals("+") || s.equals("-") || s.equals("*") || s.equals("/") || s.equals("^") || s.equals("%");
9     }
10
11     private boolean closeparenthesis() {
12         int count = 0;
13
14         for (int i = 0; i < output.length(); i++) {
15             char c = output.charAt(i);
16
17             if (c == '(') {
18                 count++;
19             } else if (c == ')') {
20                 count--;
21             }
22         }
23
24         return count > 0;
25     }
26
27     public String Check(String value) {
28         if (output.length() >= 50) {
29             return output;
30         }
31         if (output.isEmpty()) {
32             if ((isOperator(value) && !value.equals("-")) || value.equals("(")) {
33                 return output;
34             }
35         } else {
36             String last = String.valueOf(output.charAt(output.length() - 1));
37
38             if (isOperator(value) && isOperator(last)) {
39                 return output;
40             }
41             if (value.equals("(") && last.equals("(")) {
42                 return output;
43             }
44             if (value.equals("(") && isOperator(last)) {
45                 return output;
46             }
47             if (last.equals("(") && isOperator(value) && !value.equals("-")) {
48                 return output;
49             }
50             if (last.equals("(") && (value.equals("(") || value.matches("[0-9]*"))) {
51                 return output;
52             }
53             if (last.matches("[0-9]*") && value.equals("(")) {
54                 return output;
55             }
56             if (value.equals("(") && !closeparenthesis()) {
57                 return output;
58             }
59             if (value.matches("[0-9]*")) {
60                 if (output.equals("0")) {
61                     output = value;
62                     return output;
63                 }
64                 if (output.length() >= 2) {
65                     String prev = String.valueOf(output.charAt(output.length() - 2));
66                     if (last.equals("0") && (isOperator(prev) || prev.equals("("))) {
67                         output = output.substring(0, output.length() - 1) + value;
68                         return output;
69                     }
70                 }
71             }
72         }
73         output = output + value;
74         return output;
75     }
76
77     public String deleteLast() {
78         if (output.length() > 0) {
79             output = output.substring(0, output.length() - 1);
80         }
81         return output;
82     }
83 }
```

```

83
84 public String getFormattedOutput() {
85     StringBuilder result = new StringBuilder();
86     StringBuilder currentNumber = new StringBuilder();
87
88     for (int i = 0; i < output.length(); i++) {
89         char c = output.charAt(i);
90
91         if (Character.isDigit(c)) {
92             currentNumber.append(c);
93         } else {
94
95             if (currentNumber.length() > 0) {
96                 result.append(addCommas(currentNumber.toString()));
97                 currentNumber.setLength(0);
98             }
99             result.append(c);
100         }
101     }
102
103     if (currentNumber.length() > 0) {
104         result.append(addCommas(currentNumber.toString()));
105     }
106
107     return result.toString();
108 }
109
110 private String addCommas(String numStr) {
111     StringBuilder formatted = new StringBuilder();
112     int count = 0;
113
114     for (int i = numStr.length() - 1; i >= 0; i--) {
115         if (count == 3) {
116             formatted.insert(0, ",");
117             count = 0;
118         }
119         formatted.insert(0, numStr.charAt(i));
120         count++;
121     }
122     return formatted.toString();
123 }
124
125 public String getOutput() {
126     return output;
127 }
128
129 public void Clear(){
130     output = "";
131 }
132
133 public boolean readyToconvert() {
134     if (output.isEmpty()) {
135         return false;
136     }
137     String last = String.valueOf(output.charAt(output.length() - 1));
138
139     if (isOperator(last) || last.equals("(")) {
140         return false;
141     }
142
143     int openCount = 0;
144     int closeCount = 0;
145     for (int i = 0; i < output.length(); i++) {
146         if (output.charAt(i) == '(') openCount++;
147         if (output.charAt(i) == ')') closeCount++;
148     }
149
150     return openCount == closeCount;
151 }
152

```

7-6 PostFix.java

```
Source History
1 package com.mycompany.postfix;
2
3 public class PostFix {
4
5     private Stack St = new Stack(50);
6     private Process log = new Process();
7
8     private int stackPriority(char operator) {
9         if (operator == '^') {
10             return 3;
11         }
12         if (operator == '*' || operator == '/' || operator == '%') {
13             return 2;
14         }
15         if (operator == '+' || operator == '-') {
16             return 1;
17         }
18         if (operator == '(') {
19             return 0;
20         }
21         return -1;
22     }
23
24     private int inputPriority(char operator) {
25         if (operator == '^' || operator == '(') {
26             return 4;
27         }
28         if (operator == '*' || operator == '/' || operator == '%') {
29             return 2;
30         }
31         if (operator == '+' || operator == '-') {
32             return 1;
33         }
34         return -1;
35     }
36
37     public String getProcessLog() {
38         return log.getLogData();
39     }
40 }
```



```

40
41 public String toPostfix(String infix) {
42     StringBuilder postfix = new StringBuilder();
43     StringBuilder number = new StringBuilder();
44     StringBuilder visualStack = new StringBuilder();
45
46     log.startNewLog();
47
48     for (int i = 0; i < infix.length(); i++) {
49         char c = infix.charAt(i);
50
51         if (Character.isDigit(c)) {
52             number.append(c);
53         } else {
54             if (number.length() > 0) {
55                 postfix.append(number.toString()).append(" ");
56                 log.addRow(number.toString(), postfix.toString(), visualStack.toString());
57                 number.setLength(0);
58             }
59
60             if (c == '(') {
61                 St.push(c);
62                 visualStack.append(c);
63                 log.addRow(String.valueOf(c), postfix.toString(), visualStack.toString());
64             } else if (c == ')') {
65                 log.addRow(String.valueOf(c), postfix.toString() + " (Start Pop)", visualStack.toString());
66                 while (!St.isEmpty() && (char) St.top() != '(') {
67                     char popOp = (char) St.pop();
68                     postfix.append(popOp).append(" ");
69                     visualStack.deleteCharAt(visualStack.length() - 1);
70                     log.addRow(" Pop " + popOp, postfix.toString(), visualStack.toString());
71                 }
72                 if (!St.isEmpty()) {
73                     St.pop();
74                     visualStack.deleteCharAt(visualStack.length() - 1);
75                     log.addRow(" Pop (", postfix.toString(), visualStack.toString());
76                 }
77             } else {
78                 while (!St.isEmpty() && (char) St.top() != '(' && inputPriority(c) <= stackPriority((char) St.top())) {
79                     char popOp = (char) St.pop();
80                     postfix.append(popOp).append(" ");
81                     visualStack.deleteCharAt(visualStack.length() - 1);
82                     log.addRow(" Pop " + popOp, postfix.toString(), visualStack.toString());
83                 }
84                 St.push(c);
85                 visualStack.append(c);
86                 log.addRow(String.valueOf(c), postfix.toString(), visualStack.toString());
87             }
88         }
89     }
90
91     if (number.length() > 0) {
92         postfix.append(number.toString()).append(" ");
93         log.addRow(number.toString(), postfix.toString(), visualStack.toString());
94     }
95
96     while (!St.isEmpty()) {
97         char leftover = (char) St.pop();
98         visualStack.deleteCharAt(visualStack.length() - 1);
99         if (leftover != '(' && leftover != ')') {
100             postfix.append(leftover).append(" ");
101             log.addRow(" Pop " + leftover, postfix.toString(), visualStack.toString());
102         }
103     }
104
105     log.finishLog();
106
107     return postfix.toString().trim();
108 }

```

7-7 Ui.java

```
Ui.java x Process.java x StackJava [-/M] x Postfix.java x Infix.java x Main.java x Nodejava [-/A] x
Source Design History
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package com.mycompany.postfix;

/**
 *
 * @author User
 */
public class Ui extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger = java.util.logging.Logger.getLogger(Ui.class.getName());
    private Infix infix = new Infix();
    private PostFix postfix = new PostFix();

    public Ui() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    Generated Code

    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("1");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton10ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("0");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("2");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("3");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("4");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("5");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("6");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("7");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("8");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("9");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton11ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("+");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton12ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("-");
        jLabel1.setText(infix.getFormattedOutput());
    }

    private void jButton13ActionPerformed(java.awt.event.ActionEvent evt) {
        infix.Check("*");
        jLabel1.setText(infix.getFormattedOutput());
    }
}
```

```

357 private void jButton18ActionPerformed(java.awt.event.ActionEvent evt) {
358     infix.Check("%");
359     jLabel1.setText(infix.getFormattedOutput());
360 }
361
362 private void jButton14ActionPerformed(java.awt.event.ActionEvent evt) {
363     infix.Check("/");
364     jLabel1.setText(infix.getFormattedOutput());
365 }
366
367 private void jButton15ActionPerformed(java.awt.event.ActionEvent evt) {
368     infix.Check("(");
369     jLabel1.setText(infix.getFormattedOutput());
370 }
371
372 private void jButton17ActionPerformed(java.awt.event.ActionEvent evt) {
373     infix.Check(")");
374     jLabel1.setText(infix.getFormattedOutput());
375 }
376
377 private void jButton16ActionPerformed(java.awt.event.ActionEvent evt) {
378     jLabel1.setText(infix.deleteLast());
379     jLabel1.setText(infix.getFormattedOutput());
380 }
381
382 private void jButton20ActionPerformed(java.awt.event.ActionEvent evt) {
383     infix.Check("^");
384     jLabel1.setText(infix.getFormattedOutput());
385 }
386
387 private void jButton21ActionPerformed(java.awt.event.ActionEvent evt) {
388     if (infix.readyToconvert()) {
389         String infix = this.infix.getOutput();
390         String postfix = this.postFix.toPostfix(infix);
391         jLabel4.setText(postfix);
392         jTextArea1.setText(this.postFix.getProcessLog());
393     }
394     else{
395         jTextArea1.setText("The equation is incomplete.");
396     }
397 }
398
399 private void jButton22ActionPerformed(java.awt.event.ActionEvent evt) {
400     infix.Clear();
401     jLabel1.setText("");
402     jLabel4.setText("");
403     jTextArea1.setText("");
404 }
405
406 // Variables declaration - do not modify
407 private javax.swing.JButton jButton1;
408 private javax.swing.JButton jButton10;
409 private javax.swing.JButton jButton11;
410 private javax.swing.JButton jButton12;
411 private javax.swing.JButton jButton13;
412 private javax.swing.JButton jButton14;
413 private javax.swing.JButton jButton15;
414 private javax.swing.JButton jButton16;
415 private javax.swing.JButton jButton17;
416 private javax.swing.JButton jButton18;
417 private javax.swing.JButton jButton19;
418 private javax.swing.JButton jButton2;
419 private javax.swing.JButton jButton20;
420 private javax.swing.JButton jButton21;
421 private javax.swing.JButton jButton22;
422 private javax.swing.JButton jButton3;
423 private javax.swing.JButton jButton4;
424 private javax.swing.JButton jButton5;
425 private javax.swing.JButton jButton6;
426 private javax.swing.JButton jButton7;
427 private javax.swing.JButton jButton8;
428 private javax.swing.JButton jButton9;
429 private javax.swing.JLabel jLabel1;
430 private javax.swing.JLabel jLabel2;
431 private javax.swing.JLabel jLabel3;
432 private javax.swing.JLabel jLabel4;
433 private javax.swing.JPanel jPanel1;
434 private javax.swing.JScrollPane jScrollPane1;
435 private javax.swing.JTextArea jTextArea1;
436 // End of variables declaration
437 }
438

```

